

CONTENTS

- Overview
- 1 The 2023 case, and why it was convincing
- 2 What fine-tuning does to a model
- 3 The 2025 rematch
- 4 Why general models caught up
- 5 Customisation without touching the weights
- 6 Agent skills: packaging procedural knowledge
- 7 LoRA and parameter-efficient fine-tuning
- 8 Where fine-tuning still wins
- 9 The decision order
- Glossary
- Check yourself
- Where to go next

Does Fine-Tuning Still Earn Its Place?

Work out where custom weights still beat a frontier model plus a good context stack, and where they no longer do.

For engineers and leads deciding whether to invest in custom model training · 26 July 2026

Every section, key point, term and question from the full lesson is here. The extended explanations and the additional examples are not.

[Watch the source video](#)

[Read the full lesson](#)

[Read the highlights](#)

BEFORE YOU START

- What a large language model is and roughly how it is trained
- Familiarity with prompting and, ideally, RAG
- Comfort reading Python

Overview

In 2023 a well-known legal AI company fine-tuned a custom model in partnership with a frontier lab, and in blind tests attorneys preferred it over GPT-4 — the leading model at the time — 97% of the time. That looked like a settled argument. If a general model is not quite right for your domain, fine-tune it.

In 2025 the same company built a legal benchmark and tested that custom model against the latest frontier models. Seven general-purpose models, none of which had received any legal fine-tuning, had surpassed it. Bloomberg saw something comparable: they trained BloombergGPT from scratch, and later evaluations found GPT-4 and ChatGPT outperforming it on many financial benchmarks.

That is not an argument that fine-tuning is dead. It is an argument that the baseline moves, and that a custom model is a fixed artefact competing against a target that improves every few months. What changed is the size of the alternative: there is now a whole stack of customisation techniques that never touch a single weight, and most teams should exhaust it before considering training. Fine-tuning survives for a narrower and more specific set of reasons than it had in 2023.

KEY TAKEAWAYS

- ✓ Fine-tuning continues a base model's training on a focused dataset, adjusting its weights toward a narrow domain.
- ✓ A fine-tuned model is a fixed artefact competing against frontier models that keep improving — a moving target.
- ✓ Large context windows removed the main reason to bake documents into weights: you can now just supply them.
- ✓ Reasoning models shift capability to inference time, so 'how hard it thinks now' rivals 'how it was trained months ago'.
- ✓ RAG, context engineering and agent skills customise behaviour without modifying any weights.
- ✓ Agent skills package procedural knowledge as loadable files, so any general model can perform a specialised procedure.
- ✓ Most production fine-tuning today is LoRA or a related parameter-efficient method that trains a small adapter over frozen weights.
- ✓ Fine-tuning still wins for hard latency constraints, distillation into smaller models, and reinforcement fine-tuning against a programmatic grader.

01 The 2023 case, and why it was convincing

0:00

A custom legal model beat the leading frontier model in blind tests, decisively. The conclusion drawn was that domain work means fine-tuning.

KEY POINTS

- A 2023 fine-tuned legal model beat GPT-4 in blind expert testing, 97% of the time.
- The evaluation was sound: domain experts, blind, decisive margin.
- The conclusion — fine-tune when a general model is not quite right — followed reasonably.
- The comparison was against one fixed opponent at one moment in time.
- A custom model is finished; the general model it was measured against is not.

What the 2023 comparison actually established

A true statement with an implicit time bound that nobody wrote down. Reading it with the date attached makes the later result unsurprising rather than shocking.

```
custom legal model (2023)   vs   GPT-4 (2023)
  → attorneys preferred the custom model 97% of the time

# established: a fine-tune beat that specific model at that specific time
# assumed:    the comparison would keep holding
# unstated:   only one side of it was still being developed
```

02 What fine-tuning does to a model

0:41

Take a base model trained on a broad corpus and continue training it on a focused dataset, producing a new model with adjusted weights.

KEY POINTS

- Start from a pre-trained base model with broad knowledge in its weights.
- Continue training on a focused dataset of domain material not present on the open web.
- The output is a new model: base weights adjusted by the additional training.
- It is a separate artefact to host, evaluate and maintain, forked from a moving base.
- The customisation is baked into weights — not inspectable, not incrementally updatable.

The fork

The picture worth carrying. At the moment of the fork the two are equivalent plus your domain data. From then on only one of them receives further investment.

```
base model v1  — fine-tune on your data —>  your custom model
    |                                           |
    | (the lab keeps training)                  | (frozen)
    ▼                                           ▼
base model v2                                your custom model
    ▼
base model v3      ← every release widens the gap you have to re-close
    ▼
base model v4
```

03 The 2025 rematch

2:10

The same company built a legal benchmark and found seven general-purpose models, with no legal fine-tuning at all, had surpassed their custom one.

KEY POINTS

- Seven general models with no legal fine-tuning surpassed the custom model on the company's own benchmark.
- Bloomberg's from-scratch financial model was similarly overtaken on financial benchmarks.
- Fine-tuning still worked; the baseline it was measured against rose faster.
- General capability gains accrue to everyone, including those who invested nothing.
- A domain benchmark you own is the only way to detect that a custom model's lead has gone.

Two independent cases, one shape

Different domains, different organisations, different approaches — one fine-tuned, one trained from scratch. The pattern is about the baseline, not about any particular training decision.

legal AI company

2023: fine-tuned model beats
GPT-4 in 97% of blind
expert comparisons

2025: 7 general models, none
legally fine-tuned, beat
it on the company's own
benchmark

Bloomberg

trained BloombergGPT from scratch
on financial data

later evaluations found GPT-4 and
ChatGPT outperforming it on many
financial benchmarks

04 Why general models caught up

3:35

Three changes: context windows grew enormously, reasoning moved to inference time,

and inference got cheaper.

KEY POINTS

- Context windows went from around 2,000 tokens to a million and beyond.
- Knowledge that used to be baked into weights can simply be supplied in the prompt.
- Reasoning models move capability to inference time, available on any prompt without training.
- Inference keeps getting cheaper and models more efficient.
- A fine-tuning project's lead time competes with the frontier's release cadence.

What a 2,000-token window forced

The constraint that produced the 2023 consensus. When you cannot fit the material, weights are the only place to put it — which was true then and stopped being true without the advice being updated.

GPT-3 (2020)	2,000 tokens	≈ 3 pages
frontier (now)	1,000,000+	≈ 1,500 pages

in 2020: the domain corpus cannot fit → put it in the weights
now: 500 pages of contracts fit in the prompt, fresh every time,
with no training run and nothing to go stale

05 Customisation without touching the weights

5:41

RAG supplies the knowledge, and context engineering assembles everything the model needs into a deliberately constructed prompt.

KEY POINTS

- RAG retrieves documents at query time instead of training them into weights.
- Retrieved knowledge is current by construction and updated by indexing, not retraining.
- Context engineering assembles the prompt from parts, each with a defined role.
- A programmatically built prompt is versionable and testable rather than a matter of taste.
- These techniques transfer to any model and take effect immediately.

A prompt as an assembled artefact

Every block has a purpose and can be tested independently. Note the ordering: stable content first so it can be cached, variable content last.

```
def build_prompt(question, session):
    return "\n\n".join([
        SYSTEM_ROLE,                # who the model is, what it must not do
        OUTPUT_SCHEMA,              # the exact shape required back
        FEW_SHOT_EXAMPLES,         # format pinned by demonstration
        format_evidence(retrieve(question, k=8)), # RAG
        format_history(session, max_turns=6),    # trimmed state
        f"Question: {question}",
    ])

# stable blocks first → cacheable prefix
# each block independently testable → the prompt is a versioned artefact
```

06 Agent skills: packaging procedural knowledge

6:27

Folders of files that package how to do something and which tools to use, loaded on demand when the model meets a matching task.

KEY POINTS

- A skill packages procedural knowledge — how to do a task and which tools to use.
- The model loads it on demand when a task calls for it.
- RAG supplies facts; skills supply procedure. Different categories.
- A skill is a reviewable file, not an opaque weight change.
- It transfers across models and is fixed by editing rather than retraining.

A SQL skill instead of a SQL fine-tune

Everything a fine-tune would have encoded implicitly, stated explicitly and readably. When the schema changes, you edit the file — which is a pull request rather than a project.

```
skills/warehouse-sql/
├─ SKILL.md           when to use this, and the procedure to follow
├─ schema.sql         the actual table definitions
└─ conventions.md     house rules: date handling, soft deletes, naming

# SKILL.md, in essence:
# - always filter on `deleted_at IS NULL`
# - dates are stored UTC; report in Europe/Madrid
# - never SELECT *; the fact tables are wide
# - validate with EXPLAIN before running anything over 10M rows

# any capable model can follow this. no training, no artefact to host.
```

07 LoRA and parameter-efficient fine-tuning

7:11

Train a small adapter over frozen base weights. Most of what is called fine-tuning in production today is some flavour of this.

KEY POINTS

- Fine-tuning's costs are data collection, evaluation, regression avoidance and maintenance — not mainly compute.
- LoRA trains a small adapter while the base weights stay frozen.
- The update is factorised into two low-rank matrices controlled by a rank parameter.
- Adapters are small, swappable, and can be disabled — making the change reversible.
- Most production fine-tuning today is LoRA or a related PEFT method.
- LoRA cheapens the training step, not the data, evaluation or maintenance around it.

What LoRA is actually computing

The frozen weight matrix keeps its original output; the adapter adds a low-rank correction alongside it. B starts at zero, so the adapted model is exactly the base model at step zero and departs from it gradually.

```
# a normal layer:          y = W x          W is d x k, frozen
# with a LoRA adapter:    y = W x + (a/r) B A x
#
#   A : r x k    initialised random    } only these train
#   B : d x r    initialised to zero   }
#   r : the rank – the bottleneck width, and the capacity dial
#   a : alpha – scales the size of the update

# B = 0 at the start, so the adapter contributes nothing until it learns.
# trainable parameters: r(d + k) instead of d x k.
```

08 Where fine-tuning still wins

7:52

Three specific bottlenecks the no-weights stack cannot solve: hard latency limits, distillation into smaller models, and training against a programmatic grader.

KEY POINTS

- Hard latency limits: reasoning models are too slow for real-time interaction, so a small fine-tuned model wins on speed.
- Distillation: a large teacher generates training data, and a small student is fine-tuned on it.
- Distillation inverts the data problem — you generate examples instead of collecting them.
- Reinforcement fine-tuning trains against a grader rather than against fixed answers.
- RFT requires programmatic gradability — a definitive right answer you can check in code.
- Each case is a specific bottleneck, not a general preference for custom weights.

When latency is the binding constraint

A voice agent has a hard budget: past roughly a second of silence the conversation breaks down. A reasoning model that thinks for seven seconds is not slightly worse here, it is unusable — and no amount of accuracy compensates.

budget for a voice turn: < ~800 ms end to end

frontier reasoning model	2,000–8,000 ms	unusable, however accurate
frontier standard model	600–1,500 ms	marginal, variance is the problem
small fine-tuned model	80–200 ms	fits, with room for TTS

the fine-tune is not more accurate. it makes a fast model

good enough at one narrow task, which is the whole requirement.

09 The decision order

10:01

Base model, then prompt and context engineering, then RAG, then skills — and fine-tuning only for a bottleneck the rest cannot clear.

KEY POINTS

- Order: base model → prompt and context engineering → RAG → agent skills → fine-tuning.
- Each step is cheaper, faster to evaluate and easier to reverse than the next.
- The eval harness built during prompt work is what later judges the fine-tune.
- Fine-tuning requires a named bottleneck stated in one sentence.
- Knowledge gaps are RAG; procedural gaps are skills; latency and gradable outputs are fine-tuning.

The ladder, with what each rung fixes

Work down until the problem goes away. Most teams that jump to the bottom rung discover afterwards that a rung above would have been sufficient — and considerably cheaper.

1. base model	is the general model already good enough? (measure before assuming it is not)
2. prompt & context engineering	role, schema, examples, format, constraints → cost: an afternoon. reversible: instantly.
3. RAG	fresh or proprietary knowledge → cost: an index. reversible: turn it off.
4. agent skills	procedural know-how, conventions, tools → cost: a file. reversible: delete it.
5. fine-tuning	a named bottleneck none of the above clears → cost: weeks. reversible: not really.

Glossary

Base model

The off-the-shelf pre-trained model, from a frontier lab or open source, before any customisation.

Fine-tuning

Continuing a base model's training on a focused dataset so the domain behaviour ends up in the weights.

Moving target

The problem that a fixed custom model competes against frontier models that keep improving, so its lead decays without anything changing.

Context window

How much text a model can process in one call. Growth from thousands to millions of tokens removed much of the need to bake knowledge into weights.

Reasoning model

A model that performs extended step-by-step thinking at inference time, shifting capability from training to the moment of the question.

Context engineering

Deliberately assembling the prompt from parts — system role, retrieved data, examples, format rules — as a versioned, testable artefact.

Agent skill

A folder of files packaging procedural knowledge and the tools to apply it, loaded on demand by any capable model.

LoRA

Low-rank adaptation: training two small matrices whose product approximates a weight update, while the base weights stay frozen.

PEFT

Parameter-efficient fine-tuning: the family of methods, including LoRA, that adapt a model by training a small fraction of parameters.

Adapter

The small trained artefact a PEFT method produces. Swappable, stackable against one base model, and removable.

Rank (r)

The bottleneck width of a LoRA adapter, controlling its capacity and its parameter count.

Distillation

Using a large teacher model to generate high-quality outputs, then fine-tuning a smaller student model on them.

Reinforcement fine-tuning (RFT)

Training against a grader rather than fixed answers: the model samples candidates, the grader scores them, and high scores are reinforced.

Grader

A programmatic scoring function used in RFT. Requires a definitive checkable answer, which is what limits where RFT applies.

Check yourself

Answer first, then open each one.

The 2023 blind test showing 97% attorney preference was methodologically sound. Why did its conclusion still expire?

It compared a finished artefact against one snapshot of something under continuous development. The custom model stopped improving the day it shipped while the general model did not, so the measured lead was real but time-bounded — and nobody wrote the time bound down.

Why did growing context windows undermine so much of what fine-tuning was used for in 2023?

Much of that fine-tuning was knowledge injection — putting domain documents into the weights because they would not fit in a 2,000-token prompt. A million-token window does that natively, per query, with nothing to go stale and no training run. The constraint that justified the technique disappeared while the advice stayed in circulation.

What is the difference between what RAG supplies and what an agent skill supplies?

RAG supplies facts — what is true, retrieved at query time. A skill supplies procedure — how to carry out a task in this environment, in what order, with which tools and conventions. A model can have every relevant fact and still do the steps wrong, which is a skill problem and no amount of retrieval fixes it.

Why does a LoRA adapter contribute nothing to the model's output at the start of training?

Because B is initialised to zero, so the product BA is zero and the adapted model is exactly the base model. Training moves it away gradually from that known-good starting point, which is what keeps the adaptation stable rather than perturbing a working model from step one.

Why must distillation filter the teacher's outputs before training the student?

Because the student learns whatever regularity is in the data, and an unfiltered trace set contains the teacher's mistakes alongside its successes. It will reproduce both faithfully. Independent verification of each trace is what makes the generated dataset usable rather than merely large.

Why does reinforcement fine-tuning only work where outputs can be graded programmatically?

The training signal is the grader's score, so a grader that cannot distinguish good from bad optimises the model toward whatever it does reward. Where a definitive check exists — code compiles and passes tests, a query returns the right rows — you optimise the property you actually care about. Where it does not, inventing a fuzzy grader produces a model that games it.

Where to go next

- Build a domain benchmark you own for a task you care about, and score the current frontier models against any custom model you run — this is the only measurement that tells you whether a fine-tune is still earning its cost.
- Take a prompt you use in production and rewrite it as an assembled artefact with separately testable blocks, then measure the change against a fixed set of cases.
- Write an agent skill for a procedure your team performs repeatedly, and compare its results against whatever you do today for the same task.
- Train a LoRA adapter on a few hundred examples, save it, and verify for yourself that it can be loaded and unloaded against the same base model — the reversibility is the property that matters most in practice.
- Pick a task where you can write a definitive grader and estimate what fraction of your use cases meet that bar; the answer tells you how much of the RFT literature is relevant to you.
- Time your last fine-tuning project end to end and compare it against the release cadence of the models you would have been competing with.
- Read the LoRA paper (Hu et al., 2021) for the low-intrinsic-rank argument that explains why small adapters recover most of full fine-tuning's benefit.

Lesson generated from a YouTube transcript with YouTube Lesson Builder.

Explanations are AI-generated. Check anything you intend to rely on.